ПРОЕКТ • БАЗЫ ДАННЫХ

ПЛАТФОРМА ОНЛАЙН-КУРСОВ

Проектирование базы данных для платформы обучения: пользователи, курсы, уроки, платежи, сертификаты

- User
- Review
- Instructor
- Payment
- **Course**
- Certificate
- Lesson
- Message
- **Enrollment**

• 9 сущностей • 3НФ

ПРЕДМЕТНАЯ ОБЛАСТЬ

База данных платформы онлайн-обучения

В системе хранятся пользователи платформы. Пользователь может записаться на курс, оплатить его, проходить уроки, оставить отзыв, получить сертификат и общаться с преподавателем.

Управление данными

→ Пользователи и профили

→ Преподаватели (найм, специализация)

→ Курсы и уроки

→ Статус публикации

Учебный процесс

→ Запись на курсы (Enrollment)

→ Отслеживание прогресса

→ Отзывы и оценки

→ Выдача сертификатов

Монетизация и коммуникация

→ Обработка платежей

→ История транзакций

→ Возвраты (refunded)

→ Общение с преподавателем

🎯 **Центр системы: взаимодействие User ↔ Course через сущность Enrollment**

НОРМАЛИЗАЦИЯ

База данных находится в Третьей нормальной форме (3НФ)

1НФ

Первая нормальная форма

В каждой строке в каждом атрибуте содержится атомарное значение, то есть не допускаются списки через запятую и массивы. Нет повторяющихся групп.

2НФ

Вторая нормальная форма

Таблица находится в 1НФ. Для каждого составного потенциального ключа неключевые атрибуты не зависят от какого-либо подмножества этого ключа. Например, в Enrollment каждый атрибут зависит именно от пары (user_id, course_id).

3НФ

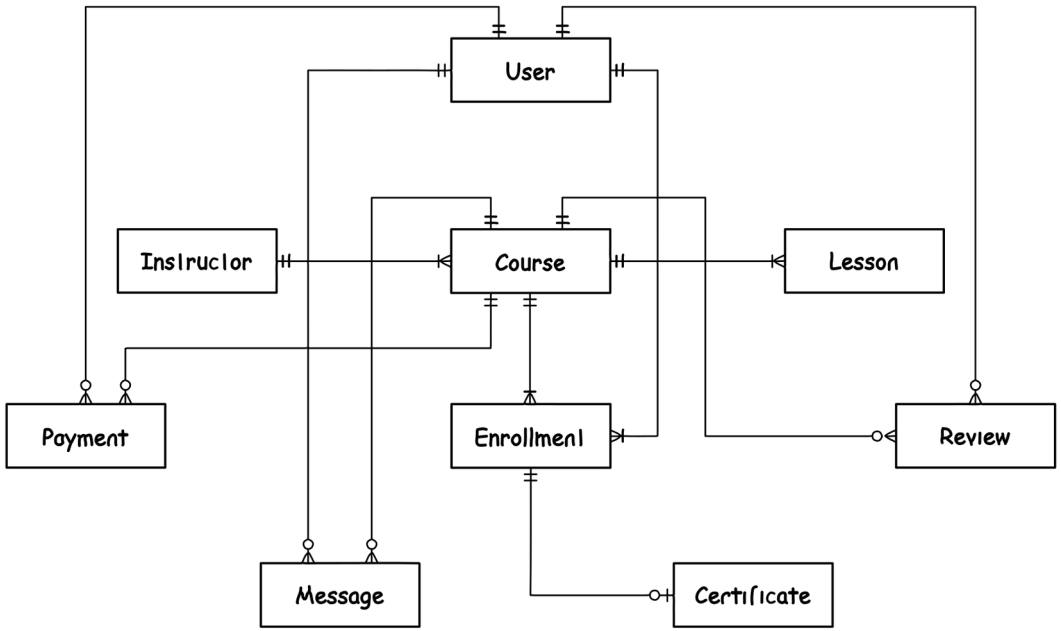
Третья нормальная форма

Таблица находится в 2НФ. Для каждого потенциального ключа отсутсвуют зависимости между неключевыми атрибутами. Например, данные преподавателя хранятся в Instructor, а в Course хранится только instructor_id. Данные курса не дублируются в Enrollment.

Замечание: избыточная нормализация может снизить производительность.

КОНЦЕПТУАЛЬНАЯ СХЕМА

ER-модель (сущности и связи)



Центр системы

User ↔ Course через Enrollment

Связь 1:M

Instructor → Course → Lesson

Связь M:N ★

User ↔ Course — промежуточная сущность Enrollment (хранит прогресс, дату, статус)

Финансы

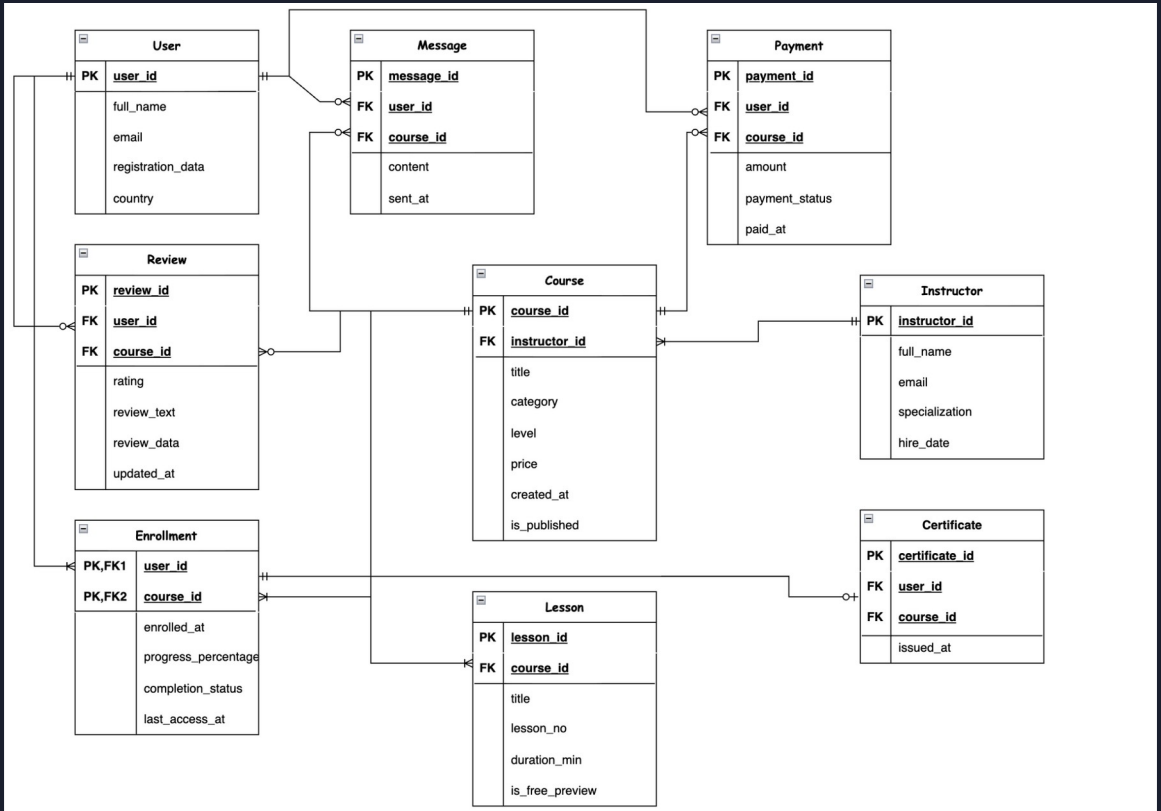
Payment — история платежей (refunded, failed...)

Итог

Certificate выдаётся при завершении (completion_status = 'completed')

ЛОГИЧЕСКАЯ СХЕМА

Таблицы, первичные / внешние ключи, ограничения



PK — Простые ключи: INT PRIMARY KEY в большинстве таблиц

★ — Составной PK: Enrollment(user_id, course_id) — пара однозначно определяет запись

FK — Внешние ключи обеспечивают ссылочную целостность по всей схеме

UQ — UNIQUE(user_id, course_id) в Review и Certificate — бизнес-ограничение на уникальность

CHK — CHECK-ограничения: progress ∈ [0,100], rating ∈ [1,5], price ≥ 0

ФИЗИЧЕСКАЯ СХЕМА

Типы данных и ограничения уровня БД

User

user_id INT PK
full_name VARCHAR(150)
email VARCHAR(255) UNIQUE
registration_data DATE
country VARCHAR(100)

Instructor

instructor_id INT PK
full_name VARCHAR(150)
email VARCHAR(255) UNIQUE
specialization VARCHAR(120)
hire_date DATE

Certificate

certificate_id INT PK
user_id INT FK → User
course_id INT FK → Course
issued_at DATE NOT NULL
UNIQUE(user_id, course_id)

Выдаётся при progress = 100%

Enrollment

user_id + course_id ← COMPOSITE PK
enrolled_at DATE NOT NULL
progress_percentage NUMERIC(10, 2)
CHECK (BETWEEN 1 AND 5)
completion_status VARCHAR(20) CHECK
last_access_at TIMESTAMP

Payment

payment_id INT PK
user_id INT FK → User
course_id INT FK → Course
amount NUMERIC(10, 2) NOT NULL , CHECK > 0
payment_ status VARCHAR(20) CHECK
(status: pending/completed/failed/refunded)
paid_at TIMESTAMP NOT NULL

Review

review_id INT PK
user_id INT FK → User
course_id INT FK → Course
rating INT NOT NULL , CHECK (BETWEEN 1 AND 5)
review_text TEXT
review_data DATE
updated_at TIMESTAMP
UNIQUE(user_id, course_id)

Все FK сопровождаются именованными CONSTRAINT — это гарантирует ссылочную целостность и упрощает отладку

ФИЗИЧЕСКАЯ СХЕМА

Lesson · Course · Message

Lesson

lesson_id INT PK
course_id INT FK → Course
title VARCHAR(200) NOT NULL
lesson_no INT NOT NULL , CHECK > 0
duration_min INT NOT NULL , CHECK > 0
is_free_preview BOOLEAN NOT NULL
UNIQUE(course_id, lesson_no)

Course

course_id INT PK
instructor_id INT FK → Instructor
title VARCHAR(200) NOT NULL
category VARCHAR(100) NOT NULL
level VARCHAR(20) NOT NULL , CHECK
price NUMERIC(10, 2) CHECK > 0
created_at DATE NOT NULL
is_published BOOLEAN NOT NULL

Message

message_id INT PK
user_id INT FK → User
course_id INT FK → Course
content TEXT NOT NULL
sent_at TIMESTAMP NOT NULL

Только при наличии Enrollment

Message: пользователь может писать только в курсе, на который записан (проверяется через Enrollment)

SQL-ЗАПРОСЫ

Аналитика платформы — фильтрация, JOIN, CASE

1 — Фильтрация курсов

```
SELECT c.course_id, c.title, c.level, c.price
FROM online_courses.course c
WHERE c.category = 'Data Science'
  AND c.is_published = TRUE
  AND c.price <= 60.00
ORDER BY c.price;
```

WHERE + AND + ORDER BY

2 — JOIN + GROUP BY + HAVING

```
SELECT c.title, i.full_name,
  COUNT(r.review_id) AS review_count,
  ROUND(AVG(r.rating), 2) AS avg_rating
FROM online_courses.course c
JOIN online_courses.instructor i ON i. instructor_id = c. instructor_id
JOIN online_courses.review r ON r.course_id = c.course_id
WHERE c.is_published = TRUE
GROUP BY c.course_id, c.title, i.full_name
HAVING COUNT(r.review_id) > 0
ORDER BY avg_rating DESC;
```

Рейтинг курсов с ≥1 отзывом

3 — CASE (популярность)

```
SELECT c.title, c.category, COUNT(e.user_id) AS enrolled_count,
  CASE
    WHEN COUNT(e.user_id) >= 5 THEN 'высокая'
    WHEN COUNT(e.user_id) >= 3 THEN 'средняя'
    ELSE 'низкая'
  END AS popularity
FROM online_courses.course c
LEFT JOIN online_courses.enrollment e ON e.course_id = c.course_id
WHERE c.is_published = TRUE
GROUP BY c.course_id, c.title, c.category
ORDER BY enrolled_count DESC;
```

Условная логика в SELECT

ПОДЗАПРОСЫ • CTE • ОКОННЫЕ ФУНКЦИИ

Сложная аналитика платформы

4 — EXISTS / NOT IN

```
SELECT u.user_id, u.full_name, u.country
FROM online_courses.user u
WHERE EXISTS (
  SELECT 1 FROM online_courses.enrollment e WHERE e.user_id = u.user_id
)
AND u.user_id NOT IN (
  SELECT cert.user_id FROM online_courses.certificate cert
)
ORDER BY u.full_name;
```

Учатся, но без сертификата

5 — LAG() • динамика по месяцам

```
WITH monthly_enrollments AS (
  SELECT DATE_TRUNC('month', enrolled_at) AS month,
         COUNT(*) AS new_enrollments
  FROM online_courses.enrollment e GROUP BY 1
)
SELECT TO_CHAR(month,'YYYY-MM') AS month,
       new_enrollments,
       LAG(new_enrollments) OVER (ORDER BY month) AS prev_month,
       new_enrollments - LAG(new_enrollments) OVER (ORDER BY month) AS diff
FROM monthly_enrollments ORDER BY month;
```

CTE + LAG() прирост/падение

6 — RANK() + CTE • топ-3 курса преподавателя

```
WITH course_revenue AS (
  SELECT c.instructor_id, c.course_id, c.title,
         SUM(p.amount) AS total_revenue,
         RANK() OVER (
           PARTITION BY c.instructor_id
           ORDER BY SUM(p.amount) DESC
         ) AS rn
  FROM online_courses.course c
  JOIN online_courses.payment p ON p.course_id = c.course_id
  WHERE p.payment_status = 'completed'
  GROUP BY c.instructor_id, c.course_id, c.title
)
SELECT i.full_name, cr.title, ROUND(cr.total_revenue, 2), cr.rn
FROM course_revenue cr
JOIN online_courses.instructor i ON i.instructor_id = cr.instructor_id
WHERE cr.rn <= 3
ORDER BY i.full_name, cr.rn;
```

RANK() + PARTITION BY

ФУНКЦИИ • ПРОЦЕДУРЫ

Бизнес-логика в базе данных

FUNCTION — get_course_completion_rate(p_course_id)

```
SELECT ROUND(
  100.0 * COUNT(cert.user_id) / NULLIF(COUNT(e.user_id), 0), 2
)
FROM online_courses.enrollment e
LEFT JOIN online_courses.certificate cert
  ON cert.course_id = e.course_id
  AND cert.user_id = e.usere_id
WHERE e.course_id = p_course_id;
```

NULLIF защищает от деления на ноль при отсутствии студентов

FUNCTION — can_user_review(p_user_id, p_course_id) → RETURNS BOOLEAN

```
SELECT
  EXISTS (SELECT 1 FROM online_courses.enrollment e
          WHERE e.user_id = p_user_id AND e.course_id = p_course_id
  )
  AND NOT EXISTS (SELECT 1 FROM online_courses.review r
                  WHERE r.user_id = p_user_id AND r.course_id =
p_course_id
  );
```

Функция — это хранимый код, который **возвращает результат**.
Процедура — это хранимый код, который обычно выполняет действие и **не возвращает результат через RETURNS**.

PROCEDURE — update_progress

```
CALL online_courses.update_progress(5, 1,
100);
```

Цепочка вызовов:
update_progress()
→ если progress = 100%
→ меняет статус на 'completed'
→ вызывает award_certificate()

Функция vs Процедура:
Функция → RETURNS, вызывается в SELECT
Процедура → CALL, управляет транзакциями

ПРЕДСТАВЛЕНИЯ

Обычное VIEW и материализованное MATERIALIZED VIEW. Представление – виртуальная таблица, являющаяся результатом запроса

VIEW — v_course_summary

```
CREATE OR REPLACE VIEW online_courses.v_course_summary AS
SELECT c.course_id, c.title, c.category,
  c.level, c.price, ...
  i.full_name AS instructor_name, ...
  COUNT(e.user_id)                  AS enrolled_count,
  COUNT(cert.certificate_id)        AS certified_count,
  COUNT(DISTINCT l.lesson_id)       AS lesson_count,
  COALESCE(ROUND(AVG(r.rating),2),0) AS avg_rating,
  COUNT(DISTINCT r.review_id)       AS review_count
FROM online_courses.course c
JOIN online_courses.instructor i ON ...
LEFT JOIN online_courses.enrollment e ON ...
LEFT JOIN online_courses.certificate cert ON ...
LEFT JOIN online_courses.lesson l ON ...
LEFT JOIN online_courses.review r ON ...
GROUP BY c.course_id, ...;
```

Не хранит результат. Пересчитывается при каждом обращении. Удобен для сводки по курсу без 5 JOIN в коде приложения.

MATERIALIZED VIEW — mv_instructor_stats

```
-- Обновляется вручную или по расписанию:
REFRESH MATERIALIZED VIEW online_courses.mv_instructor_stats;

-- Пример использования:
SELECT instructor_name, course_count, student_count,
       total_revenue, avg_rating
FROM online_courses.mv_instructor_stats
WHERE published_count > 0
ORDER BY total_revenue DESC;
```

	VIEW	MATERIALIZED VIEW
Хранение данных	Нет (запрос)	Да (снимок)
Скорость чтения	Медленнее	Быстрее
Актуальность	Всегда свежие	До REFRESH
Применение	Простая аналитика	Тяжёлые агрегации

ИНДЕКСЫ

Специальный объект БД, предназначенный в основном для ускорения доступа к данным. Вместо полного просмотра таблицы PostgreSQL сначала ищет нужные ключи в индексе, а потом переходит к нужным строкам. B-tree ДП, где каждый узел содержит множество ключей.

01

Поиск курсов по категории

```
CREATE INDEX idx_course_category_published
  ON online_courses.course
  (category, is_published);
```

Составной индекс — ускоряет самый частый запрос платформы. Без него — seq scan по всей таблице курсов.

02

История платежей пользователя

```
CREATE INDEX idx_payment_user_status
  ON online_courses.payment
  (user_id, payment_status);
```

Составной индекс — быстрый доступ к платежам конкретного пользователя с фильтром по статусу. Используется в процедуре enroll_user для проверки оплаты курса пользователем.

03

Только активные записи

```
CREATE INDEX idx_enrollment_active
  ON online_courses.enrollment
  (course_id, user_id)
  WHERE completion_status = 'active';
```

Частичный индекс — индексирует только активные строки. Меньше места, быстрее для активных студентов.

⚠ *Индекс ускоряет SELECT, но замедляет INSERT/UPDATE/DELETE — создавать только там, где есть реально медленные запросы*

ТРИГГЕРЫ

Автоматические действия при изменении данных

Триггер 1 — История enrollment (SCD Type 4)

```
CREATE TRIGGER trg_enrollment_history
  BEFORE UPDATE OR DELETE
  ON online_courses.enrollment
  FOR EACH ROW
  EXECUTE FUNCTION
    online_courses.trg_save_enrollment_history();
```

Паттерн SCD Type 4 — старая строка сохраняется в enrollment_history до изменения (UPDATE или DELETE). Тип BEFORE: OLD содержит предыдущее состояние.

Триггер 2 — Проверка права на отзыв

```
CREATE TRIGGER trg_review_eligibility
  BEFORE INSERT OR UPDATE
  ON online_courses.review
  FOR EACH ROW
  EXECUTE FUNCTION
    online_courses.trg_check_review_eligibility();
```

Бизнес-правило: нельзя оставить отзыв если не записан или бросил курс. RAISE EXCEPTION отменяет операцию. При UPDATE — автоматически обновляет updated_at.

BEFORE

Выполняется ДО изменения
Можно отменить операцию
MODIFY NEW (изменить данные)
Использовать: проверки, аудит

AFTER

Выполняется ПОСЛЕ изменения
Данные уже записаны
Каскадные действия в других таблицах
Использовать: уведомления, репликация

НАПРАВЛЕНИЯ РАЗВИТИЯ

Доработки и расширение платформы

⚡ Партиционирование

Таблица Payment по дате (paid_at) — ускорит аналитику за период, только нужная партиция читается

🎟️ Промокоды и скидки

Новая сущность Coupon, связь с Payment — монетизация и маркетинговые кампании

🔔 Уведомления

Notification: напомнить о курсе, поздравить с сертификатом — автоматически через триггеры

🔐 Роли и права

GRANT/REVOKE — аналитик без DELETE, преподаватель только свои курсы, администратор — полный доступ

🔍 Полнотекстовый поиск

tsvector/tsquery по названиям курсов и урокам — умный поиск по смыслу, не только по точному совпадению

⏰ Автоматизация

pg_cron — REFRESH материализованного представления по расписанию без ручного вмешательства

👥 Ассистенты

Instructor может делегировать проверку заданий — расширение ролевой модели платформы